

ALA – 2 - Deleting Dictionary Elements

Q1: Name and explain at least three ways to delete a specific key from a Python dictionary.

Expected Ans: Use `del dict[key]` for direct in-place removal (raises `KeyError` if missing); `dict.pop(key)` to remove and return the value (also raises `KeyError`); or `dict.pop(key, default)` to avoid errors with a fallback. Candidates should demo: `d = {'a':1}; del d['a'];` or `d.pop('a')`.

Q2: What's the difference between `del` and `pop()` for dictionary deletion?

Expected A: `del` is simpler for pure removal without needing the value; `pop()` returns it, useful for reuse, and accepts a default to handle missing keys safely. Strong answers note both are $O(1)$ average time.

Edge Cases and Errors

Q3: What happens if you try to delete a non-existent key, and how do you handle it?

Expected A: `del` or `pop()` without default raises `KeyError`. Fix with `pop(key, None)` or if key in dict: `del dict[key]`. Top candidates mention `get(key)` first to check.

Q4: Explain `popitem()` and when you'd use it.

Expected A: Removes and returns the last (arbitrary in Python 3.7+) key-value pair; useful for emptying dicts like stacks (LIFO). Raises `KeyError` on empty dicts. Example: `while d: item = d.popitem()`.

Advanced Scenarios

Q5: How do you safely delete multiple keys or while iterating? Why is direct iteration risky?

Expected A: Risk: "RuntimeError: dictionary changed size during iteration."

Solutions: Iterate over `list(dict.keys())` then delete, or use comprehension like `{k:v for k,v in d.items() if k not in keys_to_remove}`. Never mutate during for k in d:.

Q6: Delete all items matching a value (e.g., remove all keys with value 10).

Expected A: `d = {k:v for k,v in d.items() if v != 10}` (comprehension creates filtered dict). Or loop over `list(d.items())` safely. Avoids modifying during iteration.

Efficiency and Best Practices

Q7: For a large dictionary, what's the best way to delete many items? Compare to single deletions.

Expected A: Repeated `del/pop()` is $O(1)$ per op but risky if iterating; comprehensions are safer for bulk ($O(n)$ once). `clear()` empties entirely. Prefers immutability in web apps for thread safety.

